

Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

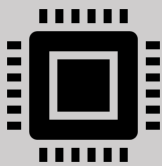
División de Ciencias de la ingeniería

Ingeniería

Organización de Lenguajes y Compiladores 2 Sección “A”

Ing. Mario Moises Ramirez Tobar

Segundo Semestre 2026



Estudiante:

Pablo Alejandro Maldonado de León

Carné: 202430233

Manual técnico Práctica #1



Descripción:

En el siguiente documento se da a conocer los detalles a tener en cuenta sobre la presente práctica. Sobre todo si se desea ampliar las funcionalidades de la aplicación se deben de conocer a detalle cómo es que funciona el traductor realizado y sobre todo las clases y los diseños que fueron aplicados para poderla desarrollar.

Disclaimer: Todo el código se encuentra escrito en ingles

“Bienvenido programador”

Como primer punto se requiere contar con los pasos para poder ejecutar el código del repositorio en la computadora local del programador, al igual que conocer la distribución de carpetas que se hizo. No sin antes mencionar todas las metodologías, tecnologías y patrones de diseño utilizados.

Tecnologías utilizadas para la realización del proyecto:

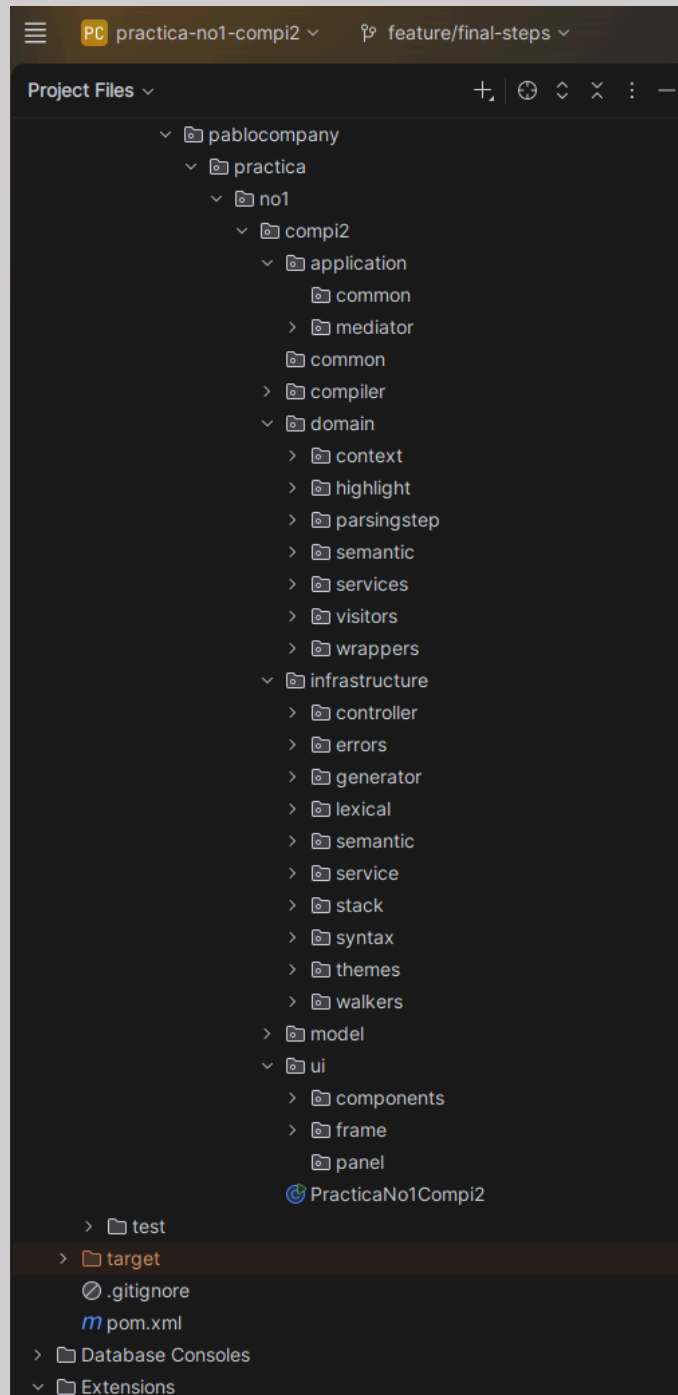
- **Java:** Utilizada como lenguaje principal sobre el que se montó la aplicación. Esta cuenta con su respectivo código dividido y organizado donde se puede interactuar tanto con el backend como en el frontend.
- **ANTLR LL(*):** Es la principal herramienta donde se creó el validador ya que permite generar una fácil integración del patrón visitor y listener para poder interactuar directamente con los diferentes estados de análisis sintáctico. Se adjunta link a la documentación oficial para saber más sobre ello:

Link: <https://www.antlr.org/>

- **Java Swing:** Es el principal motor gráfico con el que cuenta la aplicación y todos los renderizados que hace internamente están montados en esta librería.
- **Maven:** Principal gestor de dependencias del proyecto.
- **Github:** Repositorio remoto de la aplicación.

DIRECTORIO DE ARCHIVOS

Debido a que el proyecto base se realizó con cierta distribución de archivos. Así que se le recomienda al nuevo programador que mantenga el estándar que se maneja en el nombramiento de variables.



Detalles importantes:

- **La carpeta “compiler”:** es donde se encuentra toda la lógica del intérprete y de la demás lógica de negocio que maneja la aplicación.
- **La carpeta “application”:** es donde se aloja toda la capa de comunicación y puertos principales de gestión de la aplicación
- **La carpeta: “infrastructure”:** Maneja toda la lógica de negocio perteneciente a la herramienta de ANTLR y todo lo relacionado con la tecnología que conlleva.
- **La carpeta: “domain”:** Maneja todas las clases que generan la representación física de la aplicación y no depende de ninguna otra capa.

Paquete “ui”

Este paquete es el que gestiona por completo las vistas de la ui usando componentes personalizados escritos en java y sobreescritura de las propias librerías de swing. **Es importante que el nuevo desarrollador mantenga las prácticas de fragmentación en componentes. nunca generar clases enormes para tareas, siempre respetar los principios SOLID.**

IMPORTANTE:

Si se desean crear más paquetes es válido. Solo que si se debe de respetar la distribución base de los paquetes ya mencionados para seguir manteniendo el formato de orden. Sobre todo si hay que agregar documentación se le pide al programador que lo adjunte dentro del paquete llamado **documentación**.

TECNOLOGÍAS UTILIZADAS

Arquitectura de software:

Arquitectura Hexagonal

Patrones de diseño:

1. Patrón experto
2. Patrón visitor
3. Patrón Observer
4. Patrón Listener
5. Patrón Facade
6. Patrón interprete (a mínima escala sin sustituir al visitor, solo lo implementa)
7. Patrón Strategy

Paradigmas utilizados:

- Programación orientado a objetos
- Programación funcional

Principios SOLID

Otras tecnologías utilizadas:

- Modelo MVC

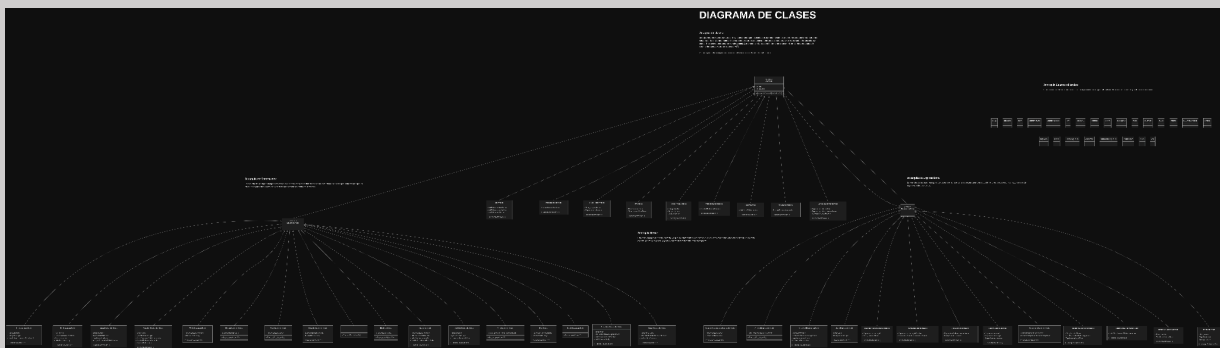
Debido a que la aplicación fue desarrollada en el entorno de Maven, IntelliJ y Netbeans, se recomienda tener instalado:

- IDE: ya sea netbeans o IntelliJ con la extensión ANTLR
- Tener instalada la versión 21 de Java
- **Tener instalada la dependencia de ANTLR4**
- **Tener instalada la dependencia de Graphviz**

DIAGRAMA DE CLASES

Este es uno de los diagramas que describe la jerarquía principal que sigue el intérprete para poder generar sus diagramas. Es una jerarquía con métodos básicos. **Cabe destacar que únicamente se delegaron los métodos por patrón intérprete que generaba el visitor. Esto permite un desacoplamiento enorme y permitió realizar las diferentes pasadas que realiza el compilador.**

Aca se deja la implementación base:



Puedes verlo más a detalle en el link:

<https://drive.google.com/file/d/1UWsbnAsko9K2daqJ40gnwmVNwBlgtKmp/view?usp=sharing>

SÍMBOLOS Y PALABRAS RESERVADAS

En el análisis léxico se tomaron las siguientes palabras como reservadas:

Se lleva a cabo un análisis léxico gracias a ANTLR que permite generar un **autómata finito determinista** en base a los principios del método de Thompson hasta poder llegar a un autómata con todos los estados que permita validar los estados de aceptación. Por lo tanto el método consiste en:

- Declaración de Expresiones regulares
- Generación de AFN **por el método de Thompson**
- Formulación de AFD en base al algoritmo de cerradura Lambda
- Minimización de estados

Nombre del archivo:

CodexLatinusLexer.g4

Palabras reservadas del lenguaje:

```
//SECTION OF CODE BLOCK
VARIABLES: 'VARIABLES';
MUNERA: 'MUNERA';
MAIOR: 'MAIOR';
FINIS_SEPARATOR: 'FINIS';

//SECTION OF FUNCTION ACTIONS
PRINT: '>';
READ: '<';

//SECTION OF VARIABLE TYPES
```

```
NUMERUS: 'numerus';

TEXTUM: 'textum';

DECIMALIS: 'decimalis';

LITTERA: 'littera';

BOOLEAN: 'bool';

//BOOLEAN VALUES

VERUM: 'verum';

FALSUS: 'falsus';

//SECTION OF SPECIAL CHARACTERS

ESTO: 'esto';

SERIES: 'series';

STRUCTURE: 'structura';

FINIS: 'finis';

DUM: 'dum';

FACERE: 'facere';

PER: 'per';

SI: 'si';

ALITER: 'aliter';

ACTIO: 'actio';

RATIO: 'ratio';

REDDERE: 'reddere';

//BREAK ACTIONS

PERGE: 'perge';
```



```
INTERRUMPE: 'interrumpe';
```

Otros símbolos entre caracteres aritméticos, usados para expresiones y creación de ámbitos:

```
//SECTION OF SPECIAL OPERATORS OR PUNCTUATION
```

```
EQUAL: '=';
```

```
COMMA: ',';
```

```
DOT_COMMA: ',';
```

```
TWO_POINTS: ':';
```

```
DOT: '.';
```

```
//SECTION OF THE GROUPING SYMBOLS
```

```
INIT_BRACE: '{';
```

```
FINAL_BRACE: '}';
```

```
INIT_BRACKET: '[';
```

```
FINAL_BRACKET: ']';
```

```
INIT_PARENT: '(';
```

```
FINAL_PARENT: ')';
```

```
//SECTION TO THE ABBREVIATION OPERATORS
```

```
ABREV_PLUS: '++';
```

```
ABREV_MINUS: '--';

//SECTION OF ARITHMETIC OPERATORS

PLUS: '+';
MINUS: '-';
MULTIPLICATION: '*';
DIVIDE: '/';

//SECTION OF RELATIONAL OPERATORS

EQUALS: '==';
GREATER_EQUALS: '>=';
LESS_EQUALS: '<=';
DIFFERENCE: '!=';
LESS: '<';
GREATER: '>';

//SECTION OF LOGIC OPERATORS

AND: '&&';
OR: '||';
NOT: 'non';
```

```
//SECTION OF VALUES
```

```
ID: [a-zA-Z_] [a-zA-Z0-9_]* ;
```

```
INT: [0-9]+ ;
```

```
DECIMAL: [0-9]+ '.' [0-9]+ ;
```

```
STRING: '"' ( '\\' . | ~["\\r\n] )* '"' ;
```

```
CHAR: '\\' ( '\\' . | ~['\\r\n] ) '\\' ;
```

En el análisis sintáctico se definieron las siguientes estructuras de terminales y no terminales:

Cabe aclarar que aquí simplemente se están agregando los **terminales** y los **no terminales**. En este caso el uso de ANTLR genera el reconocimiento automatizado de errores y el solito genera su recuperación de errores sintácticos. Basta con solo definir la gramática especificando terminales y usando los tokens reconocidos por el lexer y el solito te genera sus respectivos walkers para la fase de parseo.

Nombre del archivo:

CodexLatinusParser.g4

```
program
    : body* EOF # ProgramRoot
    ;

body:
    variable_section?
    munera_section?
    maior_section
    FINIS_SEPARATOR DOT_COMMA
    ;

/*====*****===== MAIOR SECTION =====*/
/*====*****===== MAIOR SECTION (FUNCTIONS)
=====*/

munera_section
```

```

: MUNERA GREATER munera_body # MuneraSection

;

munera_body

: munera_body functions_block # FunctionsBlockList

| functions_block          # FunctionsSingleBlock

;

functions_block

: function_declaration # FuncDeclBlock

| procedure_declaration # ProcDeclBlock

;

/*----- FUNCTION & PROCEDURE DECLARATIONS -----*/

function_declaration

: RATIO variable_function_type ID INIT_PARENT function_arguments?
FINAL_PARENT INIT_BRACE function_body? code_body? FINAL_BRACE FINIS
DOT_COMMA # FunctionDeclaration

;

/*-----Return values-----*/

variable_function_type

: variable_type          #FunctionReturNormalType

;

```

procedure_declaration

```
      : ACTION ID INIT_PARENT function_arguments? FINAL_PARENT INIT_BRACE
procedure_body? code_body? FINAL_BRACE FINIS DOT_COMMA      #
ProcedureDeclaration
```

;

function_body

```
      : VARIABLES INIT_BRACKET local_variable_list? FINAL_BRACKET      #
FunctionBody
```

;

procedure_body

```
      : VARIABLES INIT_BRACKET local_variable_list? FINAL_BRACKET      #
ProcedureBody
```

;

/----- ARGUMENTS & LOCAL VARIABLES -----*/*

local_variable_list

```
      : local_variable_list local_variable      # LocalVariablesList
      | local_variable      # LocalSingleVariable
      ;
```

local_variable

```
      : variable_declaration      # LocalVarDeclaration
      | normal_array_declaration      # LocalArrayDeclaration
```

```

| struct_instance          # LocalStructInstance
;

function_arguments
: function_arguments COMMA argument # FunctionArgsList
| argument                  # FunctionSingleArg
;

argument
: ESTO ID TWO_POINTS argument_variable_type #
ArgumentVariableDeclaration
| SERIES ID TWO_POINTS argument_series_type #
ArgumentArrayDeclarationn
;

argument_variable_type
: variable_type            # ArgumentNormalDeclaration
;

argument_series_type
: variable_type            # ArgumentArrayNormalDeclaration
;

/*==*****=====*****===== MUNERA SECTION ==*****=====*****==*/
maior_section
: MAIOR GREATER code_body # MaiorSection

```

```

;

code_body

: code_body control_block # BlockControlList
| control_block          # BlockSingleControl
;

control_block

: block_code              # BlockCode
| console_actions         # ConsoleActions
| function_call DOT_COMMA # FunctionSingleCall
| loop_control            # LoopControlAction
| return_control          # ReturnControlAction
| abbreviated_operation   # LocalAbbreviatedOperation
| variable_usage          # LocalVariableRedefinition
| array_redefined_usage   # LocalArrayRedefinedUsage
| nested_variables_usage  # LocalNestedVariableUsage
;

/*----- RETURN STATEMENT -----*/

return_control

: REDDERE expression DOT_COMMA # ReturnWithValue
| REDDERE DOT_COMMA           # ReturnVoid
;

```



```
/*----- LOOP CONTROL STATEMENTS -----*/
```

loop_control

```
: PERGE DOT_COMMA      # LoopContinue  
| INTERRUPE DOT_COMMA # LoopBreak  
;
```

console_actions

```
: nest_variable READ      # ReadVariableInput  
| READ                    # ReadInput  
| PRINT print_function DOT_COMMA # PrintAction  
;
```

print_function

```
: print_function PRINT expression # PrintMultipleExpr  
| expression                    # PrintSingleExpr  
;
```

```
/*====*****===== COMMON CODE SECTION =====*****===*/
```

block_code

```
: if_statement      # CodeBlockIf  
| while_statement   # CodeBlockWhile  
| do_while_statement # CodeBlockDoWhile  
| for_statement     # CodeBlockFor  
;
```

```
/*----- IF STATEMENT PRODUCTION -----*/
```

```
if_statement
```

```
      : SI INIT_PARENT expression FINAL_PARENT INIT_BRACE code_body?  
FINAL_BRACE else_if_list? else_statement FINIS DOT_COMMA # IfStatement  
      ;
```

```
else_if_list
```

```
      : else_if_list else_if_clause # ElseIfList  
      | else_if_clause              # ElseIfSingle  
      ;
```

```
else_if_clause
```

```
      : ALITER INIT_PARENT expression FINAL_PARENT INIT_BRACE code_body?  
FINAL_BRACE # ElseIfClause  
      ;
```

```
else_statement
```

```
      : ALITER INIT_BRACE code_body? FINAL_BRACE # ElseBlock  
      | /* Lambda */                               # ElseEmpty  
      ;
```

```
/*----- CYCLES -----*/
```

```
while_statement
```

```

        : DUM INIT_PARENT expression FINAL_PARENT INIT_BRACE code_body?
FINAL_BRACE FINIS DOT_COMMA # WhileStatement

;

do_while_statement

        : FACERE INIT_BRACE code_body? FINAL_BRACE DUM INIT_PARENT expression
FINAL_PARENT DOT_COMMA # DoWhileStatement

;

for_statement

        : PER INIT_PARENT for_init DOT_COMMA expression DOT_COMMA for_update
FINAL_PARENT INIT_BRACE code_body? FINAL_BRACE # ForStatement

;

for_init

        : ESTO ID TWO_POINTS variable_type expression # ForInitVarDecl
| ID EQUAL expression # ForInitAssign

;

for_update

        : ID ABREV_PLUS # ForUpdateIncrement
| ID ABREV_MINUS # ForUpdateDecrement
| ID EQUAL expression # ForUpdateAssign

;

/*====*****===== VARIABLES SECTION =====*****===*/

```

```

variable_section

    : VARIABLES GREATER variables_body    #VariablesSection

    ;

/*----- DECLARE VARIABLES SECTION -----*/

variables_body: variables_body declarations    # DeclarationsVariablesList
               | declarations                # DeclarationsSingleVariable
               ;

/*----- DECLARATIONS PRODUCTIONS SECTION-----*/

declarations

    : variable_declaration                # VariableInstance
    | variable_usage                      # VariableRedefinedUssage
    | normal_array_declaration            # NormalArrayInstance
    | struct_declaration                  # StructDefinition
    | array_redefined_ussage              # ArrayRedefinedUssage
    | struct_instance                     # StructVariableInstance
    | abbreviated_operation               # GlobalAbbreviatedOperation
    | nested_variables_usage              # GlobalNestedVariableUsage
    ;

```

```

/*-----VARIABLE USAGE PRODUCTIONS-----*/

/*-----STRUCT INSTANCE PRODUCTIONS-----*/

array_redefined_usage
    : ID INIT_BRACKET expression FINAL_BRACKET EQUAL expression DOT_COMMA
#RedefinedArrayUsage
    ;

variable_usage
    : ID EQUAL expression DOT_COMMA
NormalVariableRedefinedUsage
    ;

nested_variables_usage
    : struct_values EQUAL expression DOT_COMMA
#NestedStructRedefinedValue
    ;

/*-----STRUCT INSTANCE PRODUCTIONS-----*/

struct_instance
    : ESTO ID TWO_POINTS ID struct_literal
# StructInstance
    ;

/*-----VARIABLE PRODUCTIONS-----*/

```

variable_declaration

```
      : ESTO ID TWO_POINTS variable_type expression DOT_COMMA #  
VariableDeclaration  
      ;
```

normal_array_declaration

```
      : SERIES ID INIT_BRACKET expression FINAL_BRACKET TWO_POINTS  
variable_type array_initialization? DOT_COMMA # NormalArrayDeclaration  
      ;
```

```
/*_--****_-----****_-- ARRAY PROPERTIES SECTION _--****_-----****_--*/
```

array_initialization

```
      : INIT_BRACE values_array_list FINAL_BRACE # ArrayInitWithValues  
      ;
```

values_array_list

```
      : values_array_list COMMA array_value # ArrayValueList  
      | array_value # ArraySingleValue  
      ;
```

array_value

```
      : expression # ArrayNormalValue  
      ;
```

```

/*-----****--- VALUES SECTION NESTED PROPERTIES---****-----*/

struct_values

    : struct_values DOT ID                                     #
StructPropertyChain

    | struct_values INIT_BRACKET expression FINAL_BRACKET    #
StructArrayAccessChain

    | ID DOT ID                                                #
StructBaseProperty

    | ID INIT_BRACKET expression FINAL_BRACKET DOT ID        #
StructBaseArrayProperty

    ;

/*---****-----****--- STRUCT DEFINITION SECTION ---****-----*/

struct_declaration

    : STRUCTURE ID INIT_BRACE struct_body FINAL_BRACE FINIS DOT_COMMA #
StructDeclaration

    ;

struct_body

    : struct_normal_body      # StructSeparatedBody

    | struct_comma_body       # StructCommaBody

    ;

```

```

struct_normal_body
    : struct_normal_body struct_attribute DOT_COMMA      #
StructNormalBodyList
    | struct_attribute DOT_COMMA                          #
StructNormalBodySingle
;

```

```

struct_comma_body
    : struct_comma_body COMMA struct_attribute          # StructCommaBodyList
    | struct_attribute                                  # StructCommaBodySingle
;

```

```

/*-----*****----- STRUCT VARIABLES DECLARATION DEFINITION SECTION
-----*****-----*/

```

```

struct_attribute
    : variable_without_value          # NormalVariableStruct
    | array_variable_struct           # ArrayVariableStruct
;

```

```

/*-----STRUCT VARIABLE INSTANCE PRODUCTIONS-----*/

```

```

variable_without_value

```



```

: ESTO ID TWO_POINTS variable_type      # InternalStructNormalVariable
;

array_variable_struct

: SERIES ID TWO_POINTS variable_type      # InternalStructArray
;

/*-----STRUCT INSTANCE VALUES PRODUCTIONS-----*/

struct_literal

: INIT_BRACE struct_data_list FINAL_BRACE # StructLiteralValue
;

struct_data_list

: struct_data_list COMMA struct_data_value # StructValueList
| struct_data_value                        # StructSingleValue
;

struct_data_value

: ID TWO_POINTS expression                # StructDataNormal
;

/*_*****-----*****--- OPERATION SECTION ---*****-----*****_*/

```

```

expression
    :      INIT_PARENT      expression      FINAL_PARENT
# ExpressionParents

    |      op=(NOT      |      MINUS)      expression
# ExpressionUnary

    |      expression      op=(MULTIPLICATION      |      DIVIDE)      expression
# ExpressionMultDiv

    |      expression      op=(PLUS      |      MINUS)      expression
# ExpressionAddSub

    |      expression      op=(LESS      |      GREATER      |      LESS_EQUALS      |      GREATER_EQUALS)
expression      # ExpressionRelational

    |      expression      op=(EQUALS      |      DIFERENCE)      expression
# ExpressionEquality

    |      expression      AND      expression
# ExpressionAnd

    |      expression      OR      expression
# ExpressionOr

    |      normal_values
# ExpressionValue

;

/*-----*****--- VALUES AND TYPES SECTION ---*****-----*/

variable_type
    : TEXTUM      # TypeText

    | NUMERUS      # TypeInt

    | DECIMALIS # TypeDecimal

```

```

| LITTERA    # TypeChar

| BOOLEAN    # TypeBoolean

| ID         # TypeCustomId

;

/*-----*****--- ARRAY CALLING SECTION ---*****-----*/

array_call

: ID INIT_BRACKET expression FINAL_BRACKET    # ArrayCall

;

/*-----*****--- FUNCTION CALLING ---*****-----*/

function_call

: ID INIT_PARENT arguments_list? FINAL_PARENT    # FunctionCalling

;

arguments_list

: arguments_list COMMA expression            # ArgumentFunctionList
| expression                                # ArgumentSingleFunction

;

/*-----STRUCT VARIABLE INSTANCE PRODUCTIONS-----*/

nest_variable

: struct_values                # NestedValueVariable
| array_call                  # ArrayCallVariable

```

```

| ID # SigleValueVariable
;

/*-----****--- PRINCIPAL VALUES DATA ---****-----*/

normal_values

: STRING # ValString
| CHAR # ValChar
| DECIMAL # ValDecimal
| INT # ValInt
| boolean_values # ValBool
| array_call # ValArrayCall
| function_call # ValFunctionCall
| struct_values # ValStructNestValue
| struct_literal # ValStructPropertyLiteral
| array_initialization # ValArrayLiteral
| ID # ValIdCall
;

boolean_values

: VERUM # BoolTrue
| FALSUS # BoolFalse
;

abbreviated_operation

: nest_variable ABREV_PLUS DOT_COMMA # IncOperation
| nest_variable ABREV_MINUS DOT_COMMA # DecOperation

```

;

Cabe destacar algo muy importante:

Al contrario de los LALR que existen para JAVA, esta herramienta utiliza ANTLR no permite generar ambigüedad, por su mecanismo que tiene de ser un LL(*permite remover esta ambigüedad asumiendo siempre que la primera producción será de prioridad ante las demás por lo que nunca generará árboles de derivación idénticos y llegará a un estado de aceptación. Tal como pasa en las expresiones, estas tienen cierta precedencia de operaciones

```
expression
: INIT_PARENT expression FINAL_PARENT
| op=(NOT | MINUS) expression
| expression op=(MULTIPLICATION | DIVIDE) expression
| expression op=(PLUS | MINUS) expression
| expression op=(LESS | GREATER | LESS_EQUALS | GREATER_EQUALS)
expression # ExpressionRelational
| expression op=(EQUALS | DIFERENCE) expression
| expression AND expression
| expression OR expression
| normal_values
;
```

IMPORTANTE:

En este caso, ANTLR tiene un mecanismo especial para las expresiones matemáticas así que se declaran en la misma expresión y producción con un operador “|” y de esta forma se indicará que tienen la misma precedencia.

En el análisis semántico y reconocimiento de símbolos:

Se generó una implementación de tabla de símbolos con el uso de un HashMap, esto permite almacenar el valor de la variable bajo una llave única que era el identificador respectivo de la variable.

Se utilizó el mismo concepto del manejo de ámbitos, ya que como tal el único ámbito que existe es el global, se hizo uso de una implementación de pila para poder reconocer cada ámbito en su propio bloque. Lo que permite generar el reconocimiento de identificadores y el shadowing de identificadores sin generar conflictos.

Un símbolo se compone con lo siguientes atributos:

```
public class Symbol {

    //All needed propertys

    private final String id;

    private final SymbolKind kind;

    private final TypeNode type;

    private final int line;

    private final int column;

    //Scopes

    private String scope;

    private int depth;

    //This is the principal attribute property

    private boolean isInitialized;

    private boolean isUsed;

    private boolean isArray;
```

```
private Integer arraySize;

private boolean isFunction;

//For procedures or functions
private List<Symbol> parameters;
private TypeNode returnType;

//For struct types
private List<Symbol> structFields;

}
```

TABLA DE COMPATIBILIDAD DE TIPOS

Esta práctica se inspira completamente en cómo es que funciona C++ o C# ya que estos permiten el uso de expresiones en conjunto con valores booleanos y es fuertemente tipado. Cosa que para este proyecto era muy importante mantener un lenguaje fuertemente tipado pero con todo tipo de expresiones que pudieran surgir siempre bajo las restricciones que se imponen sobre la inferencia de tipos.

A continuación se deja el comportamiento de inferencia de tipos que realiza el lenguaje:

Tipo izquierdo	Tipo derecho	Operación	Tipo inferido
TEXTO	ENTERO	SUMA	TEXTO
TEXTO	CHAR	SUMA	TEXTO
TEXTO	DECIMAL	SUMA	TEXTO
TEXTO	BOOLEANO	SUMA	TEXTO
TEXTO	TEXTO	SUMA	TEXTO
DECIMAL	ENTERO	SUMA	DECIMAL
DECIMAL	ENTERO	RESTA	DECIMAL
DECIMAL	ENTERO	MULTIPLICACIÓN	DECIMAL
DECIMAL	ENTERO	DIVISIÓN	DECIMAL
ENTERO	CHAR	SUMA	ENTERO
ENTERO	CHAR	RESTA	ENTERO
ENTERO	CHAR	MULTIPLICACIÓN	ENTERO
ENTERO	CHAR	DIVISIÓN	ENTERO
ENTERO	BOOLEANO	SUMA	ENTERO
ENTERO	BOOLEANO	RESTA	ENTERO

ENTERO	BOOLEANO	MULTIPLICACIÓN	ENTERO
ENTERO	BOOLEANO	DIVISIÓN	ENTERO
CHAR	N/A	N/A	CHAR
BOOLEANO	N/A	N/A	BOOLEANO

Importante:

Cabe destacar que no se detallaron las demás expresiones ya que estas causan error y no tienen ningún valor como tal. Por lo tanto solo esas son las expresiones válidas que se permiten en el compilador.